

Kolokwium 1 - Java

Imię:

Nazwisko:

Na napisanie testu jest 85 minut. Każde poprawnie rozwiązane zadanie to jeden punkt. Każde niepoprawnie rozwiązane zadanie to zero punktów (dotyczy to również zadań z wielokrotnym wyborem)

Zanalizuj poniższe kody (zakładamy że wszystkie importy są w porządku) i:

- jeśli nastąpi błąd kompilacji klasy Zadanie, to wskaż w którym miejscu,
- jeśli klasa Zadanie się skompiluje, ale po uruchomieniu wyrzucony zostanie wyjątek to wskaż w której linii,
- jeśli kod się wykona bez wyjątku to podaj wynik uruchomienia klasy Zadanie.

1.

```
public class Zadanie {
    public static class MyException extends Exception {
        String desc;
        public MyException(String desc) { this.desc = desc; }
        public String toString() { return desc; }
    }
    public static void main(String[] args) {
        try {
            try { throw new MyException("A"); } finally { }
        } catch (Exception e) { System.out.print(e); }
        try {
            try { throw new MyException("B"); } finally { throw new MyException("C"); }
        } catch (Exception e) { System.out.print(e); }
        try {
            try { throw new MyException("D"); } finally { return; }
        } catch (Exception e) { System.out.print(e); }
    }
}
```

2. Sprawdź oba kody

○

```
import java.util.Arrays;
public class Zadanie {
    public static void main(String[] arg) {
        Object[] col = new Integer[20];
        col[17] = "Ala";
        System.out.println(Arrays.toString(col));
    }
}
```

```

    }
    ○
import java.util.ArrayList;
import java.util.Arrays;
public class Zadanie {
    public static void main(String[] args) {
        ArrayList<Object> col = new ArrayList<Integer>(
            Arrays.asList(new Integer[20]));
        col.add(17,"Ala");
        System.out.println(col);
    }
}

```

3.

```

class A {
    public Object one() { return null; }
}
class B extends A {
    public Integer one() { return new Integer(0); }
}
class C extends A {
    public String one() { return new String("s"); }
}
class D extends A {
    public Integer one() { return new Integer(0); }
    public String one() { return new String("s"); }
}
public class Zadanie {
    public static void main(String[] args) {
        System.out.println(new A().one());
        System.out.println(new B().one());
        System.out.println(new C().one());
        System.out.println(new D().one());
    }
}

```

4.

```

class A { public String test(Integer i) { return "i"; } }
class B extends A{ public String test(Object o) { return "o"; } }
public class Zadanie {
    public static void main(String[] args) {
        System.out.print(new B().test(13));
        System.out.print(new B().test("ala"));
        System.out.print(new B().test(null));
    }
}

```

```

    }
}
5.
class A {
    int i;
    A(int i) { this.i = i; }
}
class B extends A {
    B() {}
    B(int i) { super(i); }
    B(String s) { try { super(3); } finally { } }
}

```

```

public class Zadanie {
    public static void main(String[] args) {
        System.out.println(new B().i);
        System.out.println(new B(1).i);
        System.out.println(new B("2").i);
    }
}

```

```

6.
public class Zadanie {
    static {
        A a = new A();
        Zadanie.C z = new Zadanie().new C();
    }
    public static void main(String[] args) { B b = new B(); }
    static class A { static { System.out.println("A"); } }
    static class B { static { System.out.println("B"); } }
    class C { { System.out.println("C"); } }
}

```

7. Wyłumacz czym jest polimorfizm:

.....

.....

.....

.....

.....

8. Które deklaracje tablic są poprawne?

- String S[3];
- String S[] = {"a", "l", "e", "f"};
- String[] S[] = new String[6][6];
- String[] S = new String[4] {"a", "l", "e", "f"};

- `String[] T5 = new String[4];`
9. Co zostanie wypisane po uruchomieniu poniższego programu?
- ```
String s1 = "James";
String s2 = "Bond";
String s3 = s1.concat(".");
s1.concat(s2);
System.out.println(s1);
```
- James
  - Bond
  - James.Bond
  - JamesBond
  - BondJames
10. Zaznacz wszystkie poprawne zdania:
- Po każdym bloku try musi być zdefiniowany co najmniej jeden blok catch.
  - Po każdym bloku try musi wystąpić blok finally lub blok catch.
  - Po bloku try może wystąpić dowolna liczba bloków finally.
  - Jeśli blok catch i blok finally są zdefiniowane, to blok catch musi wystąpić przed blokiem finally.
11. Które ze zdań dotyczących poniższego kodu są prawdziwe?
- ```
class A {A() throws Exception {}} // 1  
class B extends A {B() throws Exception {}} // 2  
class C extends A {C() {}} // 3
```
- Klasa A dziedziczy po klasie Object.
 - Błąd kompilacji w linii 1.
 - Błąd kompilacji w linii 2.
 - Błąd kompilacji w linii 3.
12. Zaznacz wszystkie prawdziwe zdania dotyczące interfejsów
- Metody deklarowane w interfejsie są domyślnie prywatne.
 - Zmienne deklarowane w interfejsie są domyślnie public, static i final.
 - Interfejs może dziedziczyć po wielu interfejsach jednocześnie.
 - Słowo kluczowe implements oznacza, że interfejs dziedziczy po innym interfejsie.
13. Które z poniższych instrukcji wstawione w miejsce #### utworzą poprawnie instancję klasy Beta:
- ```
class Alfa {
 class Beta{ }
}
class Main {
 public static void main (String [] args) {
 Alfa a = new Alfa();
 ####
 }
}
```

- Alfa.Beta b = new Alfa.Beta();
- Alfa.Beta b = a.new Beta();
- Beta b = new a.Beta();
- Beta b = a.new Beta();

14. Które stwierdzenia dotyczące anonimowej klasy wewnętrznej są prawdziwe?

- Może dziedziczyć po dokładnie jednej klasie i implementować dokładnie jedną klasę.
- Może dziedziczyć po dokładnie jednej klasie i implementować wiele interfejsów.
- Może dziedziczyć po dokładnie jednej klasie lub implementować dokładnie jeden interfejs.
- Może implementować wiele interfejsów bez względu, czy dziedziczy po jakiejś klasie.

Zanalizuj poniższe kody (zakładamy że wszystkie importy są w porządku) i:

- jeśli nastąpi błąd kompilacji klasy Zadanie, to napisz jak poprawić kod dopisując lub usuwając tylko **jedno** słowo oraz
- jeśli teraz klasa się kompiluje ale uruchamia z błędem to wskaż przyczynę,
- a jeśli uruchamia się bez błędu to podaj wynik uruchomienia.

15.

```
public class Zadanie {
 static class A{ public A(String s) { System.out.print(s); } }
 static class B{ static A a = new A("1"); }
 static A a = new A("2");
 public Zadanie() { a = new A("3"); }
 public static main(String[] args){ }
```

}

16.

```
final class ZadanieBase {
 private int a;
 static int b = 14;
 public ZadanieBase() { a = 10; }
}
public class Zadanie extends ZadanieBase {
 public static main(String[] args) {
 System.out.println("b = " + b);
 }
}
```

17. Dla podanego kodu:

```
public class Zadanie {
 public float f(int x, float y) { return x+y; }
 public float f(float x, int y) { return x+y; }
 public float g() {
```

```

 XXX
 }
}

```

zakreśl polecenia które nie spowodują błędu kompilacji w linii XXX:

- return f(3, 4);
- return f(3.0, 4);
- return f(true, 2l);
- return f(3, 4l);
- return f(3.0, 4.0);
- return f(2/3, 3.0f);
- return f(3l, 4l);
- return f(3, 4.0f);
- return f(3.0f, 4.0f);

18. Które z poniższych deklaracji niezagnieżdżonej klasy A są niepoprawne?

- public final class A { }
- abstract final class A { }
- private final class A { }
- abstract protected class A { }
- class A { }
- final class A { }
- public static class A { }
- abstract class A { }

19. Jaki będzie wynik wykonania poniższego kodu (zakładamy, że wszystko się kompiluje)?

```

class A {
 {
 System.out.println("A1");
 }
 protected static int i = 1;
 protected static A a = new A();
 public A() { System.out.println("i = " + i); ++i; }
 static {
 System.out.println("A2");
 }
 public String toString() {
 return "A[" + i + "]";
 }
}

public class B extends A {
 {
 System.out.println("B1");
 }
 public B() { System.out.println("i = " + i); ++i; }
 {

```

```

 System.out.println("B2");
 }
 public static void main(String[] args) {
 final A a = new B();
 System.out.println(a);
 }
 public String toString() {
 return "B["+i+"]";
 }
}

```

20. Zaznacz prawdziwe zakończenia zdania “Metoda o nazwie equals w klasie A ...”:

- może mieć sygnaturę boolean equals(A a);
- musi mieć sygnaturę boolean equals(Object o); ale jako argumenty może przyjmować tylko obiekty klasy A (inaczej będzie błąd kompilacji).
- może mieć sygnaturę static boolean equals(A a1, A a2);
- może być prywatna.
- może mieć sygnaturę int equals(Object o);

21. Poniższy kod

```

class MyRuntimeException extends RuntimeException {}
public class A {
 static void f() {
 throw new MyRuntimeException();
 }
 public static void main(String[] args) {
 try {
 try {
 f();
 }
 catch (MyRuntimeException e) {
 System.out.println("MyRuntimeException");
 throw e;
 }
 catch (RuntimeException e) {
 System.out.println("RuntimeException");
 throw e;
 }
 finally {
 System.out.println("finally");
 }
 }
 catch (RuntimeException e) {}
 }
}

```

- nie kompiluje się, bo metoda f nie deklaruje, że rzuca wyjątek;

- nie kompiluje się z innego powodu;
- powoduje wypisanie:  
RuntimeException
- powoduje wypisanie:  
RuntimeException  
MyRuntimeException
- powoduje wypisanie:  
RuntimeException  
MyRuntimeException  
finally
- powoduje wypisanie:  
MyRuntimeException  
finally
- powoduje wypisanie:  
MyRuntimeException

22. Przeanalizuj poniższy kod.

```
interface I {
 public String f();
}
class A {
 public String f() {
 return "Hello!";
 }
}
class Zadanie extends A implements I {
 public static void main(String[] args) {
 I i = new Zadanie();
 System.out.println(i.f());
 }
}
```

Które z poniższych zdań są prawdziwe?

- Kod ten nie kompiluje się, ponieważ nie można tworzyć obiektów typu I.
- Kod ten nie kompiluje się, ponieważ klasa Zadanie nie implementuje metody f.
- Kod ten nie kompiluje się, ponieważ nie można napisać return "Hello!"; (poprawnie jest return new String("Hello!");).
- Kod kompiluje się, ale przy uruchomieniu zgłaszany jest wyjątek RuntimeException.
- Uruchomienie kodu spowoduje wypisanie "Hello!".

23. Wskaż które linijki skompilują się i wykonają wstawione w miejsce #### poniższego kodu:

```
public class Test {
 public static void main(String[] args) throws ClassNotFoundException {
 ####
 }
}
```



```

}
 ○ Class<String> c = String.class;
 ○ Class c1 = Class.forName("java.lang.String");
 ○ Class.forName("java.lang.String", true,
 Thread.currentThread().getContextClassLoader());
 ○ Class c2 = String.getClass();
 ○ new Class();

```

24. Jaki będzie rezultat wykonania poniższego kodu?

```

import java.util.*;
public class Subjects {
 public static void main(String[] args) {
 String[] subjects = { "Algebra", "Analiza", "Metody", "Środowisko" };
 List subjectsList = Arrays.asList(subjects);
 subjectsList.add("Dyskretna");
 subjects[0] = "Programowanie";
 System.out.println(subjectsList);
 for (String s : subjects) System.out.print(s + ", ");
 }
}

```

- [Programowanie, Analiza, Metody, Środowisko]  
Programowanie, Analiza, Metody, Środowisko,
- [Programowanie, Analiza, Metody, Środowisko]  
Programowanie, Analiza, Metody, Środowisko, Dyskretna,
- [Programowanie, Analiza, Metody, Środowisko, Dyskretna]  
Programowanie, Analiza, Metody, Środowisko,
- [Programowanie, Analiza, Metody, Środowisko, Dyskretna]  
Programowanie, Analiza, Metody, Środowisko, Dyskretna,
- Błąd kompilacji.
- Wyjątek rzucony w trakcie wykonania.

25. Jaki będzie rezultat wykonania poniższego kodu?

```

class MyPair {
 static class Pair{
 int x, y;
 public Pair(int a, int b){
 x = a;
 y = b;
 }
 }
 public static Pair f(){
 Pair x = new Pair(7, 13);
 try {
 return x;
 } finally{

```

```

 x.x = 11;
 return x;
 }
}
public static void main(String [] args) {
 System.out.println(f().x + f().y);
}
}

```

- 713
- 24
- 1113
- 20
- Błąd kompilacji.
- Wyjątek rzucony w trakcie wykonania.

26. Wybierz które z podanych poleceń można wstawić w miejsce **###** aby kod się kompilował.

```

public interface Doable {
 public void doit();
}
public class WhatToDo implements Doable {
 public void doit() {
 //...
 }
 public void createAnonymousClass() {
 new Doable() {
 public void doit() {
 ###
 }
 };
 }
}

```

- WhatToDo.doit();
- this.doit();
- super.doit();
- WhatToDo.this.doit();
- Doable.doit();
- Doable.this.doit();
- WhatToDo.super.doit();

27. Zaznacz prawdziwe stwierdzenia dotyczące usuwania obiektów w języku Java:

- Metoda finalize() usuwanego obiektu zawsze zostanie wykonana.
- Obiekt zostanie usunięty tylko wtedy gdy nie będzie istnieć żadna referencja do niego.
- Garbage collector usuwa obiekty natychmiast gdy nie są już wykorzystywane

przez inne obiekty.

- W javie istnieją tzw. słabe referencje pozwalające na dostęp do obiektów, które w każdej chwili mogą zostać usunięte przez garbage collector.
- Istnieje polecenie za pomocą, którego można wymusić uruchomienie garbage collector, jednak nie ma pewności, że wirtualna maszyna javy nie zignoruje tego polecenia.

28. Jaki będzie rezultat wykonania poniższego kodu?

```
class String {
 private final java.lang.String s;
 public String(java.lang.String s) {
 this.s = s;
 }
 public java.lang.String toString() {
 return s;
 }
}

public class Test {
 public static void main(String[] args) {
 String s = new String("Co sie stanie?");
 System.out.println(s);
 }
}
```

- Co sie stanie?
- Wystąpi błąd kompilacji.
- Przy próbie uruchomienia pojawi się komunikat o błędzie.
- Zostanie rzucony wyjątek RuntimeException.

29. Wybierz które z poniższych kodów skompilują się poprawnie:

- ```
public class A {  
    static class B extends A { }  
    class C extends B { }  
}
```
- ```
public class A {
 public A() {}
 class B extends A {
 public B() {
 super();
 }
 }
 class C extends B {
 public C() {
 super();
 }
 }
}
```

- ```

    }
    ○ public class A {
        class B extends A {}
        class C extends B {}
    }
    ○ public class A {
        class B extends A {}
        class C extends B {
            public C() {
                A.this.super();
            }
        }
    }

```

30. Jaki będzie rezultat wykonania poniższego kodu?

```

public class Test {
    public static void main(String args[]) {
        class InMain {
            public int x = 3;
        }
        Object o = (Object)new InMain();
        InMain test = (InMain)o;
        System.out.println("x = " + test.x);
    }
}

```

- x = 3
- Wystąpi błąd kompilacji.
- x = 0x13
- Zostanie rzucony wyjątek ClassCastException.